

Technical Analysis of the Main Malware Categories

NECKER

November 16, 2025

Contents

1	Introduction to Malware	2
2	Self-Replicating Malware	2
2.1	Virus	2
2.1.1	Case Study: CIH (Chernobyl Virus)	2
2.2	Worm	3
2.2.1	Botnets, Zombies, and C&C Architecture	3
3	Obfuscation and Anti-Analysis Techniques	3
3.1	Polymorphic Viruses	4
3.2	Metamorphic Viruses	4
3.3	Difficulties in Malware Reverse Engineering	4
3.3.1	Packing and Encryption (Runtime Packers)	4
3.3.2	Anti-Debugging Techniques	5
3.3.3	Anti-VM / Anti-Sandbox Techniques	5
4	Payload-Based Malware	5
4.1	Trojan (Trojan Horse)	5
4.2	Backdoor	6
4.3	Rootkit	6
4.4	Ransomware	6
4.4.1	Case Study: WannaCry	7
4.5	Spyware and Keyloggers	7
4.6	Logic Bomb	7
5	Modern Malware: Fileless	8

1 Introduction to Malware

The term **Malware** (short for *Malicious Software*) refers to any software designed with the intent to cause damage, disrupt service, steal sensitive data, or gain unauthorized access to computer systems.

From a technical perspective, a malware is a program that performs actions contrary to the interests of the user or the system owner. Its classification is not always clear-cut, as many modern threats are **hybrid** (for example, a Worm that installs a Trojan, which in turn downloads a Ransomware).

The main categories are distinguished based on two fundamental criteria:

- **Propagation Vector:** How the malware spreads (e.g., autonomous replication, human interaction).
- **Payload:** The malicious action that the malware executes once activated (e.g., data theft, encryption, concealment).

2 Self-Replicating Malware

This category includes software specifically designed to spread.

2.1 Virus

A **Virus** is a fragment of code that **requires a host file** to propagate. It is not a *standalone* (independent) program. It "hooks" onto a legitimate executable (e.g., `.exe`, `.dll`, scripts) or a boot sector (Boot Sector).

- **Lifecycle:** The virus remains dormant until the host file is executed. At that point, the viral code is executed first, searches for other host files to infect, and finally passes control to the legitimate program (so as not to arouse suspicion).
- **Infection:** It can occur in various ways:
 - *Appending/Prepending:* Adds its code to the beginning or the end of the host file.
 - *Cavity (Empty space):* Inserts itself into unused spaces (e.g., padding) within the sections of a PE (Portable Executable) file.
 - *Entry-Point Obfuscation (EPO):* A program has the address of the first instruction to execute in its header; instead of modifying this address (which would make the virus easy to detect), it modifies the entry point of the PE to point to itself (i.e., it places a jump instead of the first instruction), but does so in an obfuscated way to evade static analysis, making the code extremely difficult to read and full of useless jumps here and there.

2.1.1 Case Study: CIH (Chernobyl Virus)

CIH (known as "Chernobyl") is a clear example of a highly destructive virus (active at the end of the 1990s).

- **Type:** PE file infector virus (Windows 9x).
- **Technique:** It exploited "cavities" in executable files, making detection difficult through simple file size checks.

- **Payload:** On April 26 (the anniversary of the Chernobyl disaster), the virus would activate. Its payload was dual and catastrophic:
 1. It overwrote the first megabyte of the hard disk (overwriting the MBR (**Master Boot Record**: a partitioning scheme that defines how the disk is subdivided and contains the initial code to boot the operating system) and the FAT (**File Allocation Table**: a file system that manages the organization of files inside a single partition)), rendering the system unbootable and the data unrecoverable.
 2. It attempted to overwrite the **Flash BIOS** of the motherboard. If successful (on compatible chipsets), it rendered the hardware unusable (requiring physical reprogramming of the chip).

2.2 Worm

Unlike a virus, a **Worm** is a *standalone* program that actively replicates across the network. It does not need a host file or human interaction (after the initial infection) to spread.

- **Propagation:** It exploits known vulnerabilities (for example, a buffer overflow in a network service) or transport mechanisms (email, network shares) to copy itself onto other systems and self-execute.
- **Impact:** Its primary objective is spreading. The main impact is the saturation of network bandwidth. However, modern worms almost always carry a secondary payload (such as a backdoor or ransomware).

2.2.1 Botnets, Zombies, and C&C Architecture

The most common use of worms today is the creation of **Botnets**.

- **Zombie (or Bot):** A system infected by the worm. The system is now under the control of a remote operator (the *bot-herder*).
- **Botnet:** The entire network of infected "zombie" computers.
- **Command & Control (C&C or C2):** The infrastructure (one or more servers) used by the bot-herder to send commands to the botnet.

C&C architectures can be:

- **Centralized (IRC/HTTP):** All bots connect to a specific server or IRC channel. Easy to manage, but vulnerable (if the C&C server is taken down, the botnet dies).
- **Decentralized (P2P):** Bots communicate with each other, exchanging commands. Much more resilient to a *takedown*.

Botnets are "rented out" to perform large-scale malicious activities, such as **DDoS** (Distributed Denial of Service) attacks, sending spam/phishing, mining cryptocurrencies, or acting as proxies for other illicit activities.

3 Obfuscation and Anti-Analysis Techniques

To avoid detection by signature-based antivirus (AV) software and to complicate analysis by researchers, malware employs advanced obfuscation techniques.

3.1 Polymorphic Viruses

A polymorphic virus (or malware) alters its own code to look different with each infection, while maintaining the exact same functionality.

- **Mechanism:** The main body of the malware (*payload*) is encrypted with a different key each time.
- **Polymorphic Engine:** The malware consists of two parts:
 1. The **encrypted payload**.
 2. A small **decryption routine** (called a *decryption stub* or *loader*) that is executed first.
- **Mutation:** With each new infection, the polymorphic engine generates a new key, re-encrypts the payload, and, most importantly, **mutates the decryption routine** (e.g., by inserting NOP instructions, changing the instructions used, reordering the code).
- **Detection:** Static signatures are ineffective (because the payload is encrypted and the stub changes). It requires **heuristics** or **emulation** (the AV runs the code in a sandbox, waits for it to decrypt itself, and analyzes the payload in memory).

3.2 Metamorphic Viruses

Metamorphism is an evolution of polymorphism. A metamorphic malware **does not encrypt its own body**, but instead **completely rewrites its own code** with each propagation.

- **Mechanism:** It uses an integrated **metamorphic engine** (effectively a mini-compiler/assembler).
- **Mutation Techniques:**
 - *Register Swapping:* Swaps the registers used (e.g., `MOV EAX, 1` becomes `MOV EBX, 1` and uses `EBX` thereafter).
 - *Instruction Substitution:* Replaces instructions with equivalents (e.g., `SUB EAX, EAX` to clear a register, instead of `XOR EAX, EAX` or `MOV EAX, 0`).
 - *Code Transposition:* Reorders blocks of code and functions, linking them back together with `JMP` and `CALL`.
 - *Garbage Code Insertion:* Inserts "garbage" code that does nothing (NOP instructions or useless calculations) to alter the binary signature.
- **Detection:** Extremely complex. Since there is no static payload to decrypt, emulation is less effective. It requires semantic code analysis or N-gram analysis techniques to identify statistical patterns.

3.3 Difficulties in Malware Reverse Engineering

Reverse engineering (static analysis with disassemblers like IDA Pro, Ghidra, and dynamic analysis with debuggers like x64dbg, OllyDbg) is the main task of a malware analyst. Modern malware is specifically designed to counter this activity.

3.3.1 Packing and Encryption (Runtime Packers)

Many malware samples are "packed". The original executable code is compressed and/or encrypted and embedded as a resource or data section inside a *loader* (stub).

- **Static Analysis:** Useless. The disassembler only sees the loader code (e.g., UPX, Themida, VMProtect, or custom packers).
- **Dynamic Analysis:** The analyst must run the malware inside a debugger, reach the Original Entry Point (OEP) – the point where the code has been completely decompressed in memory – and perform a *memory dump* to obtain the original payload to work on.

3.3.2 Anti-Debugging Techniques

The malware actively tries to detect the presence of a debugger.

- **API Checks:** Calls functions like `IsDebuggerPresent()` or `CheckRemoteDebuggerPresent()`.
- **Timing Attacks:** Measures the execution time of certain instructions (e.g., RDTSC). If the execution is too slow (due to breakpoints and debugger step-over), the malware terminates.
- **Exception Handling:** Exploits how debuggers handle exceptions (e.g., SEH - Structured Exception Handling). The malware can intentionally generate an exception and place its malicious code inside the handler, knowing that a debugger will intercept the exception before the handler does.

3.3.3 Anti-VM / Anti-Sandbox Techniques

The malware tries to determine whether it is running inside a virtualized environment (VMWare, VirtualBox) or a sandbox (Cuckoo, ANY.run).

- **Virtual Artifacts:** Looks for the presence of specific files, drivers, or registry keys (e.g., `VBoxGuestAdditions.sys`, `vmtoolsd.exe`).
- **MAC Address:** Checks the MAC address of the network card (VMs have known OUIs: 08:00:27 for VirtualBox, 00:05:69 for VMWare).
- **I/O Port Checks:** Uses privileged instructions or I/O ports utilized by virtualization drivers to communicate with the hypervisor.
- **Systems "Too Clean":** Checks browser history, recent files, or the number of CPU cores. An analysis VM is often too "clean" or has few resources.

If it detects an analysis environment, the malware **terminates execution** or **executes a benign payload** (e.g., a "decoy") to deceive the analyst.

4 Payload-Based Malware

This category focuses on the final *purpose* of the malware.

4.1 Trojan (Trojan Horse)

A **Trojan** is a malware that masquerades as legitimate, useful, or desirable software (e.g., a game, an utility, a software update) to induce the user into executing it.

- **Operation:** Unlike viruses and worms, a **Trojan does not self-replicate**. It relies on social engineering.
- **Purpose:** It is a **delivery vector** (*dropper* or *downloader*). Once executed, it secretly installs other types of malware.
- **Common Types:**

- **RAT (Remote Access Trojan):** Provides the attacker with full remote access to the system (like a hidden VNC or RDP session).
- **Banking Trojan:** (E.g., *Zeus*, *TrickBot*). Injects code into web browsers to steal banking credentials (often via *Man-in-the-Browser*).
- **Downloader:** A small Trojan whose sole purpose is to download and execute a larger payload (e.g., ransomware) from the C&C server.

4.2 Backdoor

A **Backdoor** is a mechanism that bypasses normal authentication controls to secure persistent and hidden access to a system for the attacker.

It can be a specifically installed program (often by a Trojan or a Worm) or an intentional weakness left in legitimate software by a developer (for maintenance or malicious purposes). A RAT is an advanced form of backdoor.

4.3 Rootkit

A **Rootkit** is software (or a set of tools) specifically designed to **conceal the presence** of an attack (e.g., hiding files, processes, network connections, and the presence of the rootkit itself). It obtains elevated privileges (often at the *root* or *kernel* level).

- **User-Mode Rootkit:** Operates in user space. It uses techniques such as:
 - **API Hooking (IAT/EAT Hooking):** Modifies the import/export tables (Import Address Table / Export Address Table) of system libraries (e.g., `kernel32.dll`). When a legitimate program (e.g., Task Manager) attempts to list processes, its API call is intercepted, and the rootkit filters its own process out of the list.
 - **DLL Injection:** Injects its own DLL into all active processes to control them from within.
- **Kernel-Mode Rootkit:** Operates at the kernel level (Ring 0). It is much more powerful and dangerous.
 - **Malicious Drivers (LKM in Linux):** Loads as a legitimate device driver.
 - **DKOM (Direct Kernel Object Manipulation):** Directly modifies opaque kernel data structures. For example, to hide a process, it removes its node (`EPROCESS`) from the linked list of active processes maintained by the kernel.
 - **SSDT Hooking:** Modifies the *System Service Descriptor Table*, which maps user-mode API calls to their kernel-mode implementations.
- **Detection:** Kernel-mode rootkits are nearly impossible to detect from within the infected operating system (since they control the APIs used to detect them). They require offline scanners (run from a boot CD/USB) or memory forensics.

4.4 Ransomware

A **Ransomware** is a malware that monetizes the attack through extortion.

- **Locker Ransomware (obsolete):** Blocks access to the system (e.g., shows a full-screen message) but does not encrypt data. Easily bypassed.
- **Crypto-Ransomware (modern):** Encrypts the user's files (documents, photos, databases) using robust cryptographic techniques (e.g., a combination of AES-256 and RSA-2048).

Infection Process (Crypto-Ransomware):

1. **Delivery:** Via phishing, exploit kits, or worms.
2. **Key Generation:** The malware contacts the C&C server to obtain an RSA public key or generates a key pair on the spot (encrypting the private key with a master public key).
3. **Encryption:** For speed, the malware generates a symmetric key (e.g., AES-256) for each file. It encrypts the file with AES and then encrypts the AES key with the attacker's RSA public key. The result (encrypted AES key) is saved inside the file or in a metadata file.
4. **Backup Destruction:** Searches for and deletes shadow copies (`vssadmin delete shadows /all /quiet`) and other local backups.
5. **Ransom:** Drops a note (*ransom note*) explaining that the files are encrypted and that the only way to recover them is to pay a ransom (usually in Bitcoin) to obtain the RSA private key (or a decryptor).

4.4.1 Case Study: WannaCry

WannaCry (2017) was a hybrid attack that combined a **Ransomware** with a **Worm**.

- **Propagation Vector (Worm):** Utilized the **EternalBlue** exploit (stolen from the NSA and leaked by the Shadow Brokers group). This exploit targeted a critical vulnerability (MS17-010) in the Windows **SMBv1** protocol.
- **Mechanism:** Once a system was infected, WannaCry encrypted files (Ransomware payload) and simultaneously scanned the network (both LAN and Internet) searching for other vulnerable Windows machines on port 445 (SMB) to spread (Worm payload).
- **Impact:** Its worm nature allowed it to spread globally in a few hours, hitting hundreds of thousands of computers, including critical systems such as hospitals (NHS in the UK), transport, and corporate logistics.
- **Kill Switch:** The attack was slowed down by the discovery of a "kill switch" (a hardcoded domain that the malware tried to contact; if the connection succeeded, the malware terminated execution).

4.5 Spyware and Keyloggers

Spyware is software designed to collect information about the user, their habits, or their credentials without consent.

- **Keylogger:** A specific type of spyware that records every keystroke pressed by the user. It is the most direct method to steal passwords, conversations, and credit card data. It can be implemented via software (hooking the `GetAsyncKeyState` APIs) or hardware (a physical device between the keyboard and the PC).
- **InfoStealer:** Steals saved information (e.g., passwords and cookies from browsers, configuration files of FTP clients, or cryptocurrency wallets).

4.6 Logic Bomb

A **Logic Bomb** is a malicious code fragment intentionally inserted into software, which remains dormant until a specific **trigger condition** occurs.

- **Trigger:** Can be a specific date or time (e.g., Friday the 13th), the occurrence or non-occurrence of an event (e.g., running a program), or the presence/absence of a file (e.g., a file checking if an employee is still active in corporate systems).
- **Context:** They are almost always the work of **insiders** (e.g., a disgruntled systems administrator preparing a bomb to wipe servers if their account is disabled).
- **Difference from Payload:** Unlike a virus payload (such as CIH, which activates on a specific date), a logic bomb is typically a targeted attack and not designed for replication.

5 Modern Malware: Fileless

A critical trend in malware evolution is the **Fileless** approach (or "Living-off-the-Land" - LotL). These attacks attempt to avoid writing any malicious executable files to the disk, thereby eluding traditional antivirus scanning (which relies on file analysis).

- **Mechanism:** The malware operates entirely **in-memory**, meaning it never downloads the virus file but loads it directly into **RAM** so as not to be detectable.
- **Vectors:** It exploits legitimate tools already present in the operating system to enter the system:
 - **PowerShell:** Extremely powerful. It can download scripts from the network and execute them directly in memory (`IEX (New-Object Net.WebClient).DownloadString('...')`).
 - **WMI (Windows Management Instrumentation):** Can be used for command execution and persistence (WMI event subscriptions).
 - **Macros (VBA):** Malicious macros in Office documents that download and execute scripts in memory.
 - **System Registry:** Used to save the payload, which is then read and executed by another script.
- **Detection:** Highly difficult. It requires advanced behavioral monitoring (EDR - Endpoint Detection and Response), memory analysis, and granular logging of executed scripts (e.g., PowerShell Script Block Logging).